Look-up tables

CMPT 145

Learning Objectives

After studying this chapter, a student should be able to:

- Describe what a Table is, in terms of its possible applications.
- Explain how a variation of the treenode data structure can be used to implement the Table ADT.
- Explain how the binary search tree can be used to make the Table ADT implementation efficient.

Table ADT

- An application of Binary Search Trees
- A Table provides search, insertion, and deletion of keyed data
- A `key` is a value that is unique to the data being stored, e.g.,
  - Student number
  - International Standard Book Number (ISBN)
  - Social Insurance Number
- A key tells us which data we're looking for, but the data may be much more than the key, e.g.,
  - Student record, employee record
  - Data about a book, or the entire contents
  - A health record, or a taxation record

KVTreeNode Class

```
1  class KVTreeNode(object):
2      def __init__(self, key, value, left=None, right=None):
3          """
4          Create a new KVTreeNode for the given data.
5          Pre-conditions:
6              key:    A key used to identify the node
7              value:  Any data value to be stored in the KVTreeNode
8              left:   Another KVTreeNode (or None, by default)
9              right:  Another KVTreeNode (or None, by default)
10         """
11         self.value = value
12         self.left = left
13         self.right = right
14         self.key = key
```

Table ADT

- Purpose:
  - Manage a keyed data
- Implementation:
  - Data:
    - keys and values
  - Essential Operations:
    - Create empty table
    - Query if table is empty
    - Query size of table
    - Insert key, value into table
    - Delete key, value from table
    - Retrieve the data for a given key

Table Class: An Object Oriented ADT

```
1 class Table(object):
2     def __init__(self):
3         self.__root = None
4         self.__size = 0
```

Table Class: An Object Oriented ADT

```
1     def retrieve(self, key):
2         """
3         Return the value associated with the given key.
4         Preconditions:
5             :param key: a key
6         Postconditions:
7             none
8         Return
9             :return: True, value if the key appears in the table
10                     False, None otherwise
11        """
```

Table Class: An Object Oriented ADT

```
1          def retrieve_prim(tnode):
2              if tnode is None:
3                  return False, None
4              else:
5                  ckey = tnode.key
6                  if ckey == key:
7                      return True, tnode.value
8                  elif key < ckey:
9                      return retrieve_prim(tnode.left)
10                 else:
11                     return retrieve_prim(tnode.right)
12
13         return retrieve_prim(self.__root)
```